DeepSeek-V4.1-Flash 预训练/后训练解析

📌
前置推荐阅读：

- DeepSeek-V4.1-Flash 架构：技术报告/源码解析
- DeepSeek-V4 训练流程: Pretrain+Online Distillation
- Muon 优化器
- NEW On-Policy Distillation 是什么？工作脉络梳理
- 2026 年基座模型中的 On-Policy Distillation 技术总结

📌
技术报告：《DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression》
模型仓库：deepseek-ai/DeepSeek-V4.1-Flash
包含知识点：

- **预训练：**45T 文本 + 多模态混合语料、直接训练 sparse attention（取消 DeepSeek V4 的 dense 冷启动）、DeepSeek-ViT 通过 contrastive pre-training + autoregressive fine-tuning 两阶段预训练。
- **后训练：**复用 V4 后训练 Pipeline，不引入新的后训练算法，投入在数据和环境 Scaling。包括大规模 Agent 任务-环境合成、Scaffold + Compute 的异步 RL Scaling、40+ Teacher full-vocabulary OPD。

1. 预训练（Pre-Training）

> 前置阅读：DeepSeek-V4 训练流程: Pretrain+Online Distillation。
> DeepSeek V4 的数据配方、稳定性手段与 OPD 框架在上文已完整解析，下面主要关注 V4.1-Flash 的改进。

V4.1-Flash 的预训练相对 V4 有三处结构性变化：

- 语料扩充到 45T token（文本与多模态混合）。
- 视觉编码器先单独两阶段（2-Stage）训练、再接入进行统一预训练。
- Embedding 参数切换优化器。

1.1 训练数据

1.1.1 文本语料

- **Scaling ladder**：在模型参数与训练数据维度上进行 Scaling 实验指导大规模训练的配比。
- **隐式重复（implicit duplication）**：过滤弱模型输出与低质量机翻文本。
- **代码时效**：补入新发布的开源仓库、commit、库与框架，覆盖更多编程语言。

1.1.2 多模态语料

多模态预训练数据分图文 pair、图文交错、领域数据三类，V4.1-Flash 论文认为网页原生数据已含丰富的多模态知识，因此明确不做大规模合成，直接清洗使用。

- **爬虫重置**：初始爬取系统偏向纯文本网页，改从 Common Crawl 重新 bootstrap。
- **图文 pair**：抽取网页图片与 alt text，按图文相关性阈值过滤，按图像语义去重。
- **领域数据**：visual grounding 与 pointing、OCR、长尾知识，以及 image-code 对与 computer-use 轨迹。

图文交错数据主要来自网页与 PDF，处理成本高于纯文本，因此按成本递增分三级处理：

1. 拉取图片之前，用启发式与统计过滤、去重、质量模型挑出高价值文档。
2. 组装成图文交错序列，再做图像感知的过滤与去重。
3. 用 SmolVLM 对图文内容严格打分，被筛掉的文档部分回收成图文对。

1.1.3 Merge / Packing

- 文本与多模态重叠样本用多模态版本替换纯文本版本，替换后纯文本与多模态的 token 比是 7:1。
- 预训练与上下文扩展阶段联合预取并分配样本，尽量减少样本重叠。
- 超长文档在混合前确定性切分，使训练 token 在各数据分片与训练步之间分布均匀。
- 预训练使用 sample-level attention masking。

1.2 训练配置

整个预训练过程中学习率、序列长度与 ViT 冻结状态随 Token 训练数量的设置：



- 稀疏注意力去掉冷启动：
  - DeepSeek V4 先用 dense attention 适应 64K 上下文，再做 indexer warmup，最后切换稀疏注意力。
  - V4.1-Flash 直接在 64K 上下文训练稀疏注意力。
- 稳定性部分去掉 DeepSeek V4 的 Anticipatory Routing。
- 沿用 DeepSeek V4 的 SwiGLU Clamping。
- 64K 的上下文扩展到 1M 的上下文需要 YaRN 外推，外推倍数 `rope_factor = 16`：`compress_ratio` 为 0 的层（第 0、1 层与 DSpark 层）不使用外推，用基础 rope_theta。

1.3 优化器

> 推荐前置阅读：Muon 优化器

1.3.1 参数分组优化

参数组	优化器	超参数
RMSNorm 权重、bias、缩放因子等非矩阵参数	**AdamW**	$\beta_1 = 0.9$、$\beta_2 = 0.95$、$\varepsilon = 10^{-20}$、weight decay 0.1。weight decay 只施加在 norm 权重上，bias 与缩放因子不施加。
主干线性变换权重、Engram 的 projection layers、vision-language projector	**Muon**	momentum 0.95、weight decay 0.1，用 Nesterov momentum 与 decoupled weight decay。每个更新矩阵的 RMS 缩放到 0.18，以复用 AdamW 的 LR 经验值。
Query/Key 权重	**head-wise Muon**	正交化按 head 分别做。
全部 Embedding 与 prediction head	**Sinkhorn-balanced update**	Nesterov momentum、不施加 weight decay，LR 修正系数同 Muon，$K = 11$、$\tau = 10^{-3}$、$\varepsilon = 10^{-20}$。Engram 的 LR 放大 5 倍。

1.3.2 Sinkhorn-Balanced Update

Embedding 这类大矩阵参数的优化器保留 Muon 的骨架，把 Newton-Schulz 正交化替换为 Sinkhorn balancing (交替做行归一与列归一，逼近双随机矩阵)：

$$\Delta_t = \sqrt{n} U^{(K)} = \sqrt{n} D_r \widehat{G}_t D_c,$$

$$\frac{1}{n}\sum_{j=1}^{n} (\Delta_t)_{ij}^2 \approx 1, \qquad \frac{1}{m}\sum_{i=1}^{m} (\Delta_t)_{ij}^2 \approx 1$$

- $W_t \in \mathbb{R}^{m \times n}$ 是参数矩阵，embedding table 和 output head 的 m 是 vocab size，n 是 hidden dim。
- $\widehat{G}_t$ 是 Nesterov momentum 更新，$U^{(K)}$ 是 K 步交替归一后的矩阵：奇数步按行、偶数步按列除以 ℓ_2 范数。
- D_r、D_c 是归一化累积出来的对角缩放矩阵，$\sqrt{n}$ 负责把逐元素均方拉回 1 的量级。

Algorithm 1 Momentum update with Sinkhorn balancing

Require: Weight $W_t \in \mathbb{R}^{m\times n}$, gradient G_t, momentum M_{t-1}, momentum coefficient β, base learning rate η_t, learning-rate correction γ, numerical constants ε and τ, and an odd number of normalization steps K

$M_t \leftarrow \beta M_{t-1} + (1-\beta)G_t$
$\widehat{G}_t \leftarrow \beta M_t + (1-\beta)G_t$ ▸ Nesterov momentum
$\rho_i \leftarrow \|\widehat{G}_{t,i,:}\|_2$ and $\bar{\rho} \leftarrow \frac{1}{m}\sum_{i=1}^{m}\rho_i$
$U^{(0)} \leftarrow \widehat{G}_t$
Set $U^{(0)}_{i,:} \leftarrow 0$ if $\rho_i \leqslant \tau\bar{\rho}$ ▸ Mask near-zero rows
for $k = 1, \ldots, K$ **do**
  if k is odd **then**
    for all $i = 1, \ldots, m$ **do**
      $U^{(k)}_{i,:} \leftarrow U^{(k-1)}_{i,:} / (\|U^{(k-1)}_{i,:}\|_2 + \varepsilon)$
    end for
  else
    for all $j = 1, \ldots, n$ **do**
      $U^{(k)}_{:,j} \leftarrow U^{(k-1)}_{:,j} / (\|U^{(k-1)}_{:,j}\|_2 + \varepsilon)$
    end for
  end if
end for
$\Delta_t \leftarrow \sqrt{n}\,U^{(K)}$ ▸ Convert unit row ℓ_2 norm to unit row RMS
$\bar{\eta}_t \leftarrow \gamma\eta_t$ ▸ Match the update magnitude of Adam
$W_{t+1} \leftarrow W_t - \bar{\eta}_t\Delta_t$

本质上这是用行列双随机的近似替换正交化。Newton-Schulz 要求整个矩阵接近正交，代价随 m（vocab size）增长。Sinkhorn 只要求行方向、列方向的均方各自约为 1，$K = 11$ 步即可收敛，显存开销只存 momentum。

1.4 视觉编码器 DeepSeek-ViT 训练

> 前置阅读：SigLIP

DeepSeek-ViT 在接入语言主干之前单独两阶段训练：对比预训练（Contrastive pre-training）、自回归微调（Autoregressive fine-tuning）：



1.4.1 对比预训练（Contrastive pre-training）

- 损失是 SigLIP 的 sigmoid 对比损失，数据是约 47B image-text pairs。
- 输入分辨率上限 224×224，更大的图等比缩小。

> 分辨率上限压到 224×224 的实测依据：论文观察到这一阶段用更高分辨率有明显收益，但在最终模型的收益很小，提高对比阶段的分辨率于是只增加计算开销，整体改进有限。

同期 Kimi K3 训练流程解析：多模态预训练 + 后训练（冷启动 Agentic SFT + 多专家 RL + MOPD） 的 MoonViT-V2 走相反路线，不做 SigLIP 式对比预训练，直接用语言建模损失从头训练。

1.4.2 自回归微调（Autoregressive fine-tuning）

- 把第一阶段的 ViT 接入 4B MoE LLM，用 next-token prediction 在 236B token 上训练，数据包含 image caption / alt text / chart / OCR。
- 输入分辨率限定在 544×544 到 1344×1344 之间，超界图片等比缩放，后续预训练沿用同一策略。

这一阶段的目标是增强 encoder 对细粒度视觉特征的建模能力。训练结束后丢弃 LLM，只保留 encoder 进入 pre-training，分辨率策略保持不变。

1.4.3 接入 pre-training

ViT 接入之后与 LLM 在同一个预训练里训练，ViT 一直冻结到 LR 衰减阶段，期间只有末层 norm 与 vision-language projector 保持可训。衰减一开始就解冻 ViT，用更小的学习率与 LLM 联合优化。

1.5 预训练评估结果

在内部评测框架、统一设置下对比三个 base 模型：

	Benchmark (Metric)	# Shots	DeepSeek-V4-Flash Base	DeepSeek-V4-Pro Base	DeepSeek-V4.1-Flash Base
	Architecture	-	MoE	MoE	MoE
	# Activated Params	-	13B	49B	8B/16B
	# Backbone Params	-	284B	1.6T	552B
World Knowl.	AGIEval (EM)	3-5-shot	83.9	**84.4**	83.4
	MMLU-Pro (EM)	5-shot	68.3	73.5	**74.1**
	C-Eval (EM)	5-shot	92.1	**93.1**	92.1
	MultiLoKo (LLM-Judge)	5-shot	42.6	**50.9**	45.5
	Simple-QA verified (EM)	25-shot	30.1	**55.2**	42.3
	SuperGPQA (EM)	5-shot	46.5	**53.9**	53.1
Lang. & Reas.	BBH (EM)	3-shot	86.9	**87.5**	86.1
	BBEH (EM)	1-shot	25.4	**29.8**	27.2
	DROP (F1)	1-shot	**88.6**	**88.7**	87.9
	HellaSwag (EM)	0-shot	85.7	**88.0**	87.2
Code & Math	BigCodeBench (Pass@1)	3-shot	56.8	59.2	**60.6**
	HumanEval (Pass@1)	0-shot	69.5	76.8	**79.4**
	GSM8K (EM)	8-shot	90.8	92.6	**93.0**
	MATH (EM)	4-shot	57.4	**64.5**	61.1
	MGSM (EM)	8-shot	**85.7**	84.4	80.2
Long Context	LongBench-V2 (EM)	1-shot	44.7	**51.5**	45.2
Multimodal	MMMU-Pro (EM)	4-shot	-	-	56.5
	CVBench (EM)	4-shot	-	-	77.9
	DocVQA (LLM-Judge)	4-shot	-	-	95.6
	RefCOCO-avg (Acc@0.5)	0-shot	-	-	86.0

重点看 V4.1-Flash-Base（8B/16B 激活）对比 V4-Pro-Base（49B 激活）：

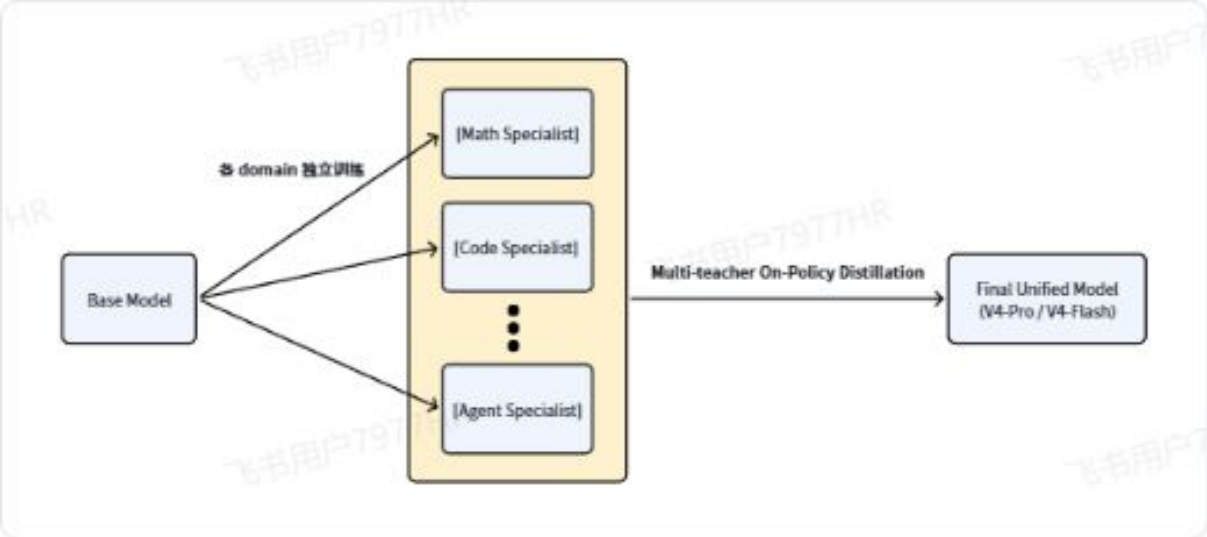
- V4.1-Flash-Base 的激活参数远少于 V4-Pro-Base，KV cache 也大幅缩减，整体表现仍然持平：MMLU-Pro

74.1 vs 73.5、SuperGPQA 53.1 vs 53.9、HumanEval 79.4 vs 76.8。这意味着这一版的收益不只来自 token scaling，data pipeline 的改动在参数效率上同样有贡献。

- 但知识维度上对 V4-Pro 仍有明显差距，Simple-QA verified 42.3 vs 55.2、MultiLoKo 45.5 vs 50.9，说明知识容量这部分主要还是由激活参数量决定。

2. 后训练（Post-Training）

> **推荐前置阅读：**
>
> 1. NEW On-Policy Distillation 是什么？工作脉络梳理
> 2. DeepSeek-V4 训练流程: Pretrain+Online Distillation



DeepSeek V4 后训练 Pipeline

DeepSeek-V4.1-Flash 的 post-training 仍然使用 DeepSeek-V4 的 SFT / RL / OPD 后训练路径，**论文明确声明不引入新的后训练算法**，投入几乎全部放在训练数据上而不是优化方式上：

- 合成 diverse / verifiable 的训练任务，连同 reference solution 与 reward signal 产出。
- 程序化构建并扩展 interactive agent environment（Agent 能真实交互、结果可自动判定的环境），把 trajectory 的采集与评估成本压低。
- 严格的 filtering、dedup、difficulty calibration，保证数据质量 / curriculum 平衡。

> V4.1 给出的判断是：在固定且普通的优化流程下，合成数据与环境在 scale / diversity / verifiability 上的系统性改进解释了几乎全部增益，这条路线的边际回报高于后训练的算法创新。

2.1 大规模 Agent 任务合成

每个任务形式化成一个三元组 `(problem, environment, verification system)`，质量由两个维度决定：

- **难度（difficulty）**：任务不能是平凡的。
- **正确性（correctness）**：三个组件不含致命缺陷。

关键设计点：任务每进入一次新的 RL run，产出的 trajectory 反过来成为重新审计该任务的证据，质量控制因此覆盖任务的整个生命周期。

2.1.1 General Agent

General Agent 的环境全部来自数据回收：

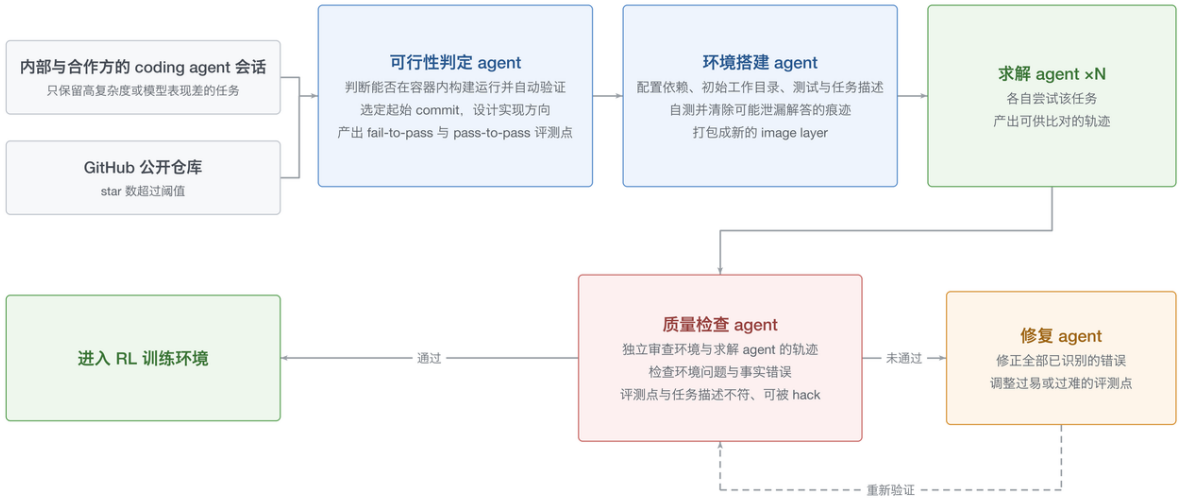
- **数据来源**：内部员工与外部合作方自愿把最新模型接入日常 workflow，回传交互数据与反馈。
- **Mock 工具**：按回传数据里出现过的接口，复现真实工具的输入格式、输出结构、API schema、行为约束，覆盖常用 SaaS、企业应用与专门的业务后端系统。
- **失败回放**：大规模收集 negative feedback 与失败案例，重建当时的 tool context、用户交互模式、失败条件，生成 single-turn 与 multi-turn 环境，针对已观测到的弱点做 RL。

2.1.2 Coding Agent

Coding Agent 的环境来自两个来源：

- 内部与外部合作 coding agent session (过滤后只保留高复杂度或模型表现差的任务，再按 trajectory dedup)。
- star 数超过阈值的公开 GitHub repo。

构建环节由多个分工不同的 agent 依次协作完成，包括可行性判据、打包前清除解答痕迹、独立于求解的质量检查：



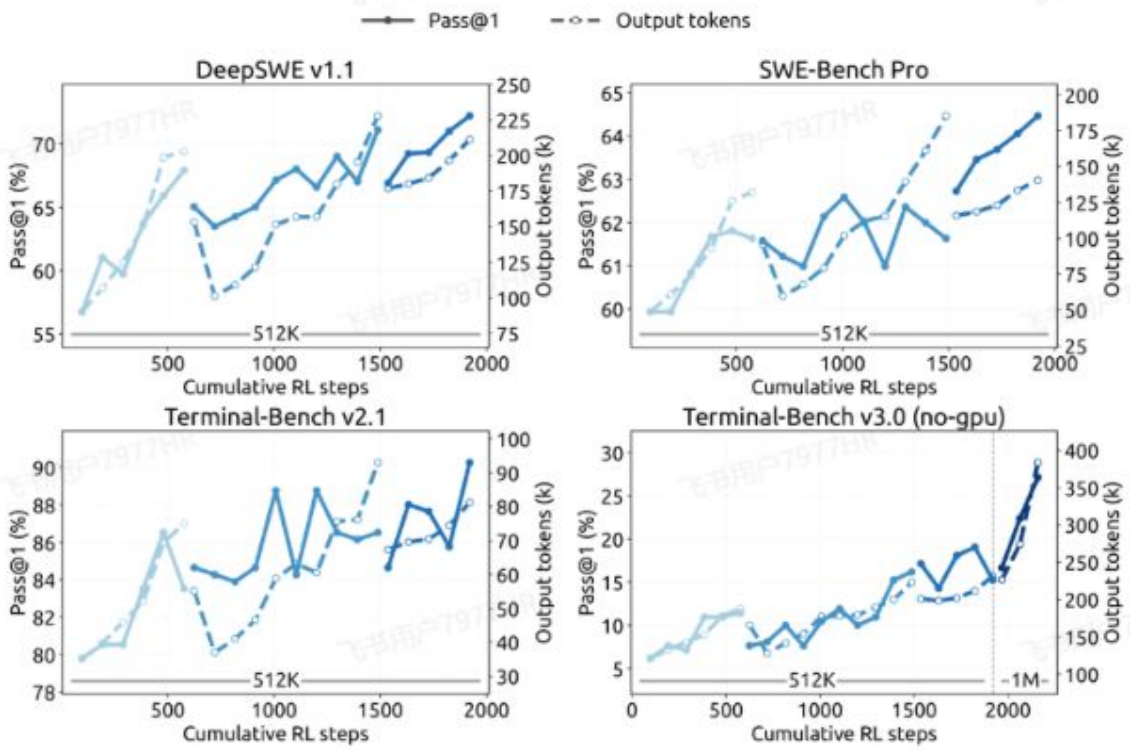
1. **可行性判定 agent**：判断项目能否在容器内完整构建运行、能否自动验证。通过之后选定起始轮次或 commit，设计若干足够复杂的实现方向，产出 `fail-to-pass` 与 `pass-to-pass`，必要时联网获取外部资源。
2. **环境搭建 agent**：在隔离容器内配置依赖、初始工作目录、测试代码与任务描述，先行自测，清除一切可能泄漏解答的痕迹，再打包成新的 image layer。
3. **N 个求解 agent**：各自尝试该任务，为质量检查提供可供比对的轨迹。
4. **质量检查 agent（quality-inspection agent）**：独立审查环境本身与求解 agent 的 trajectory，检查环境问题、事实错误、评测点与任务描述不符，以及 hackability 风险。
5. **修复 Agent**：不通过则交给修复 agent，修正全部已识别的错误、调整过易或过难的评测点，重新进入验证。

2.2 合成任务上的 RL

2.2.1 RL Scaling：训练算力 / scaffold 数量

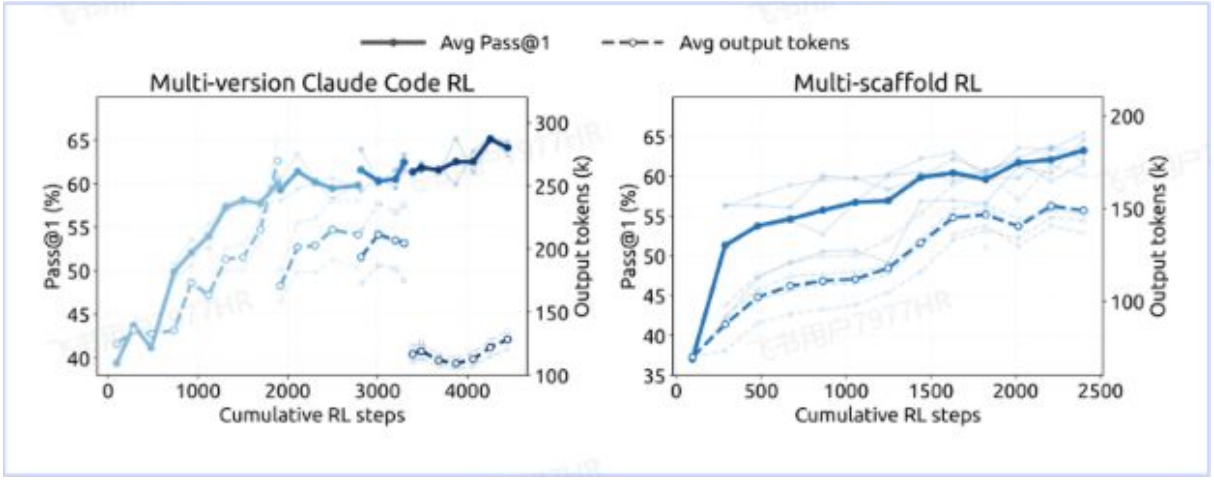
大规模异步 RL 在两个维度上扩展，training compute 和 scaffold（即外层 agent 框架，如 Claude Code、Codex）的数量。

- 单个 scaffold 内（DeepSeek Harness 的 Minimal 模式），四个 code benchmark 随 RL steps 的变化：



横轴下方标着上下文预算，前三段 run 的最大上下文都是 512K，Terminal-Bench v3.0 在最后一段扩到 1M 之后继续上升。

- 跨版本与跨 scaffold 的联合训练，左边是多个 Claude Code 版本，右边是 OpenCode、Pi 与 DeepSeek Harness 的 Standard、PTC 模式：



2.2.2 Model Merging

动机：单次 RL run 能利用的有效算力有限，单个 run 训练到一定 step 之后继续加算力，收益递减。

实现：用 model merging 初始化下一轮 RL，把不同 scaffold 或不同配置的 run 的 checkpoint 合并，汇总沿不同优化路径得到的改进。上面两张图里断开的曲线段，就是 reinitialization 之后的后续 run，合并在 task performance 与 token efficiency 两侧都有进一步提升。

2.3 DSec（DeepSeek Elastic Compute）

> 推荐前置阅读：执行与沙箱（Execution）

DSec（DeepSeek Elastic Compute）是 V3 到 V4 期间建设的 Agentic 训练与评测 sandbox 平台，V4.1 的 Agentic RL 训练需求达到数百万并发实例，瓶颈随之转移到 datacenter scalability、workload isolation、per-node density，以及 Agent 越界行为的遏制（containment of misbehavior）。

2.4 Controllable Reasoning Effort

动机：大模型存在 cost–quality trade-off，DeepSeek V4 的做法是训练并保存三个不同 RL 配置的 checkpoint 来提供 Non-think / Think High / Think Max 三档思考预算，档位离散保存权重。

实现：V4.1 在 RL 里引入标量 effort level $b \in \{1, \dots, 100\}$ 作为显式的 conditioning signal。以一行指令写在 system prompt 开头，single-turn reasoning 与 multi-turn agentic 任务都使用：

因此单 checkpoint 由此可以沿学到的 cost–quality frontier 移动，训练只用有限个档位。

2.4.1 分组采样与长度惩罚

> **这里要解决的问题：**如何让同一个模型学会区分档位，同时又不让不同档位的回答互相比较（GRPO 的组相对优势不能混入 effort，否则优势计算会失去意义）。

每个训练 prompt 在每个 effort 档位上各采样 M_b 条回答：

$$z_{b,j} \sim \pi_\theta(\cdot \mid x, b), \quad b \in \mathcal{B}, \quad j = 1, \dots, M_b$$

- x 是训练 prompt，$\mathcal{B}$ 是训练使用的有限档位集合，j 索引同一档位下的回答。
- 相同 (x, b) 的回答构成一个 subgroup，reward 在 subgroup 内部 mean-centered 得到 group-relative advantage，不同档位的回答不直接比较。档位之间的行为差异不靠跨档位比较产生，而靠 reward 里的 length penalty 项随 b 变化来诱导：

$$r^{\text{len}}_{b,j} = -\min\left\{C_{\max},\ k(b)\frac{\ell_{b,j}}{L_{\text{norm}}}\right\},$$
$$k(b) = k_0 \exp\left(-\frac{b - b_{\min}}{\tau}\right), \qquad \tau = \lambda\overline{\Delta b}$$

- $\ell_{b,j}$ 是 reasoning token 数量，L_{norm} 是参考长度，$C_{\max}$ 是单条 trajectory 扣分的上限。
- k_0 是最低档 $b_{\min}$ 处的 basic penalty coefficient，$\overline{\Delta b}$ 是训练档位的平均间距，λ 控制衰减速率。
- b 每增加 τ，$k(b)$ 就乘以 e^{-1}。k_0 决定整体压短推理的力度，τ 越小惩罚衰减越快。

2.5 异步后训练 Infra（Asynchronous Post-training Infrastructure）

动机：rollout 阶段的长尾是 RL 训练效率的主要瓶颈。

实现：rollout 与 training 在同一批物理设备上 colocate 并分时执行。每个任务只指定 in-flight 样本数上限，不需要手动在两个阶段之间分配资源。完整循环：



2.5.1 dispatch 粒度

dispatch 粒度最终使用 sample-level，V4.1 消融测试过其他两种粒度：

- **Batch-level**：起始阶段多派发几个 batch，每轮训练之后补满一个 batch，结果训练指标剧烈震荡，说明这个粒度太粗。
- **Prompt-level**：一个 GRPO 组完成之后派发一个新 prompt。结果是容易卡在组内的长尾样本上，难以平稳维持目标并发。
- **Sample-level（最终方案）**：新完成的样本数一旦达到下一个 prompt 的 GRPO group size，就立刻派发该 prompt，无论这些样本来自哪些组，并发全程平稳。

2.5.2 Length Bias 与 Off-Policy

异步生成带来两个副作用分开处理：

- **Length bias（长度分布偏置）**：短序列先完成，会主导训练早期的 batch。dispatcher 按数据集限制 concurrency 调节稳态配比，再丢弃过早返回的短样本。
- **Off-policy**：部分或全部 token 由更早的 checkpoint 生成。调整 dispatch 逻辑与等待条件给最大 off-policy ratio 设上界，训练时再加 loss masking 剔除陈旧度过高的 token。

需要 Infra 支持 token-level 中断，训练所需数据集齐之后在途样本立即停止。KV cache 与专家路由按 token 粒度持久化，换 checkpoint 后直接复用，省掉 re-prefill 开销。

2.6 大规模 OPD（Large-Scale On-Policy Distillation）

后训练的最后阶段是 full-vocabulary OPD，在全部 domain 的数据上训练，同样采用 asynchronous generation。方法脉络见 2026 年基座模型中的 On-Policy Distillation 技术总结。

- **40 个以上的 teacher**：各 domain 的训练过程不同，每个 domain 的最佳 teacher 可能来自模型的不同阶段。
- **架构可异构**：teacher 之间、teacher 与 student 之间的架构允许不同。post-training infra 支持 teacher 数量不设上限，并以可忽略的代价在它们之间切换。
- **训练中动态重配（dynamic reconfiguration）**：持续跟踪模型能力并相应调整配方，包括 dataset mixture、各数据集的 concurrency limit、以及当前 active 的 teacher。

3. 实验

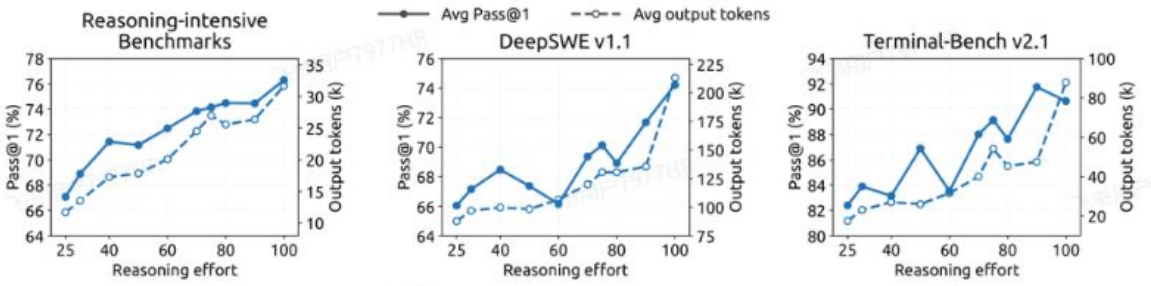
3.1 Post-Training 模型主实验

	Benchmark (Metric)	Opus-5 Max	GPT-5.6 Sol Max	K3 Max	GLM-5.3 Max	DS-V4-Pro Max	DS-V4-Flash Max	DS-V4.1-Flash Max
Reasoning	GPQA Diamond (Pass@1)	93.4	**94.1**	92.9	88.1	92.4	89.9	90.9
	HLE (Pass@1)	**56.3**	44.5	43.5	42.0†	42.7†	37.8†	36.8 (39.1†)
	Codeforces (Rating)	-	-	-	-	3348	3289	**3471**
	MathArena Apex (Pass@1)	-	-	**65.6**	-	65.3	58.6	**65.6**
Agentic	Terminal-Bench 2.1 (Pass@1)	89.1	88.8	88.3	88.2	87.9	82.7	**90.6**
	Terminal-Bench 3.0 (Pass@1)	**43.3**	34.4	17.7	28.3	11.8	7.6	30.0
	Terminal-Bench 4.0 (Pass@1)	**51.8**	39.9	12.6	37.9	12.4	7.0	31.2
	DeepSWE v1.1 (Resolved)	74.0	73.0	67.5	66.9	62.7	54.4	**74.2**
	ProgramBench (Almost@1)	**37.0**	23.0	17.5	19.0	15.5	-	20.3
	NL2Repo-Bench (Score)	**75.3**	56.8	58.0	58.0	61.5	54.2	65.4
	CyberGym (Pass@1)	-	84.5	80.0	84.5	83.3	76.7	**88.1**
	SEC-Bench Pro (Pass@1)	-	**74.3**	-	-	56.4	30.9	62.8
	ExploitGym (Pass@1)	22.1	**33.7**	-	15.0	5.4	1.8	15.3
	HLE w/ tools (Pass@1)	63.6	-	59.8	62.5	60.0	51.5	**63.9**
	Automation-Bench (Pass@1)	50.3	45.8	46.7	48.8	43.2	37.7	**54.8**
	Agents' Last Exam(Pass@1)	28.6	26.7	27.6	28.5	25.7	25.2	**31.8**
	Chartography w/ tools (Pass@1)	**84.0**	79.9	68.1	-	-	-	78.9
	BabyVision w/ tools (Pass@1)	**94.1**	88.9	85.7	-	-	-	89.6
	ZeroBench-main w/ tools (Pass@5)	52.0	**53.0**	41.0	-	-	-	49.0

- **Coding Agent 是增量最大**：在 DeepSWE v1.1 是 74.2，Terminal-Bench 2.1 是 90.6。
- **Terminal-Bench 3.0 / 4.0 仍然落后**：在新 agentic benchmark 仍然落后闭源模型。
- **网络安全能力**：CyberGym 88.1，SEC-Bench Pro 62.8 仍低于 GPT-5.6 Sol 的 74.3。

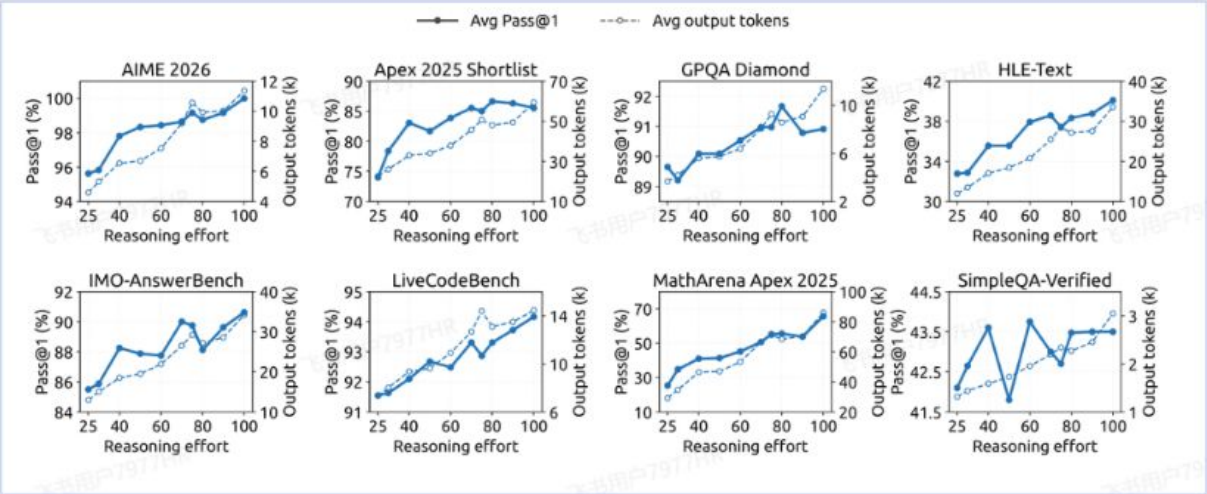
3.2 Reasoning Effort 的边际收益

Reasoning Effort 从 25 提升到 100，Pass@1 与 output length 同步上升：



收益前置（front-loaded）：Reasoning Effort 从 60 提升到 80 这一段已经恢复到最高档大部分的 accuracy，继续提升到 100 会让 agent trajectory 长出 1.6 ~ 1.8 倍，只换来边际提升。这意味着日常 Agentic 使用取中等档位更划算，最高档留给最难的任务。

在 single-response reasoning 上学到的 effort 控制同样迁移到了 long-horizon agentic trajectory，八个推理 benchmark 的逐项曲线：

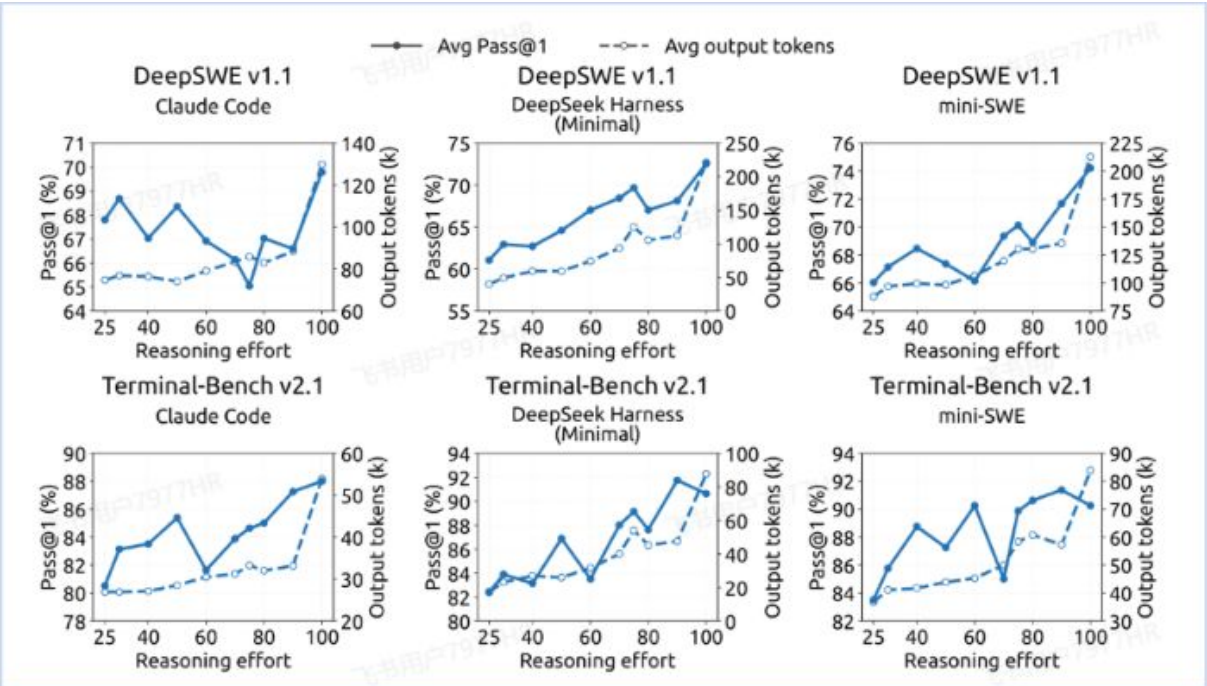


3.3 Scaffold 适配鲁棒性

模型接入到 agent 框架里，不同的 system prompt、tool definition、context management、interaction protocol 可能造成模型能力可能大幅退化。这组实验保持模型 checkpoint、decoding 配置、任务集完全一致，只换外层 harness，覆盖六个 scaffold 家族的八个配置：

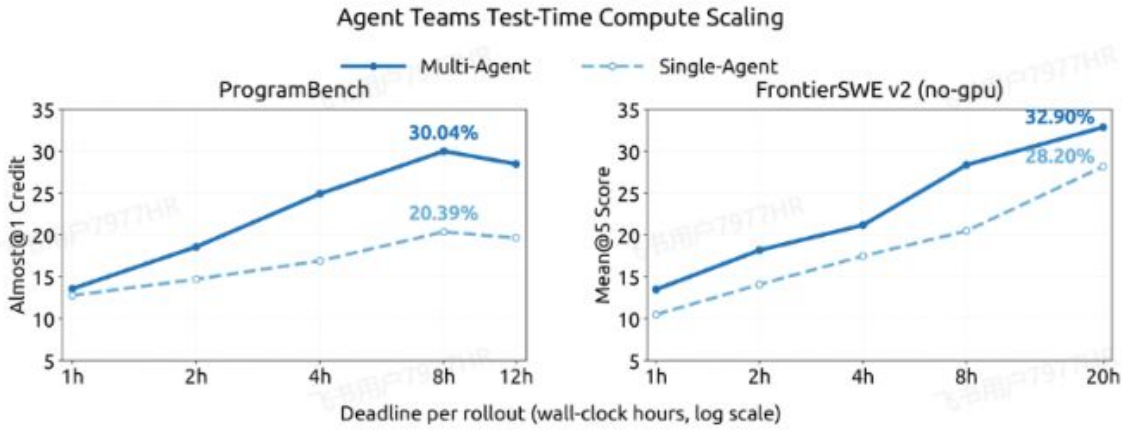
Benchmark (Metric)	Claude Code	Codex	OpenCode	Pi	mini-SWE	DeepSeek Harness		
						Minimal	Standard	PTC
DeepSWE v1.1 (Resolved)	69.8	65.6	65.5	66.2	74.2	72.6	70.5	67.6
Terminal-Bench v2.1 (Pass@1)	88.0	84.1	85.0	86.1	90.3	90.6	85.8	85.8

- DeepSWE v1.1 落在 65.5 到 74.2 之间，八个配置全部使用 Max effort（100）。
- 两项的最高分并不在同一个 scaffold 上：DeepSWE 最高是 mini-SWE 的 74.2，Terminal-Bench 最高是 DeepSeek Harness 的 90.6。
- effort 在不同 scaffold 上的表现：



3.4 Multi-Agent：Agent Team 实验

- **组织方式**：lead agent 通过 `spawn_teammate` 异步创建具名的持久 teammate，全部 agent 共享一份 repository checkout，改动彼此立即可见。
- **奖励构成**：task performance，加上鼓励 delegation 与 inter-agent communication 的 collaboration bonus，再减去 derived-latency penalty。
- **Derived latency**：把执行事件与协作依赖表示成有向无环图（DAG），按固定 prefill、decode 速率从 token 数折算成本，加上实测的 tool execution time，取 critical path 长度。鼓励有效并行、惩罚无谓的串行与同步，同时对服务端的 batching 与排队延迟不敏感。



- ProgramBench 的高置信子集（参考解在隐藏测试集上通过率不低于 95% 的 172 道题），multi-agent 的 Almost@1（得分不低于 0.95 的 rollout 占比）从 1 小时的 13.59% 升到 8 小时的峰值 30.04%，single-agent 同期是 12.79% 到 20.39%。
- FrontierSWE v2 的 no-GPU 子集（不需要 GPU 的那部分任务）上，multi-agent 的 Mean@5 从 1 小时的 13.50% 升到 20 小时的 32.90%，single-agent 是 10.50% 到 28.20%。

两个 benchmark 上 multi-agent 在每一个 deadline 上都优于 single-agent。